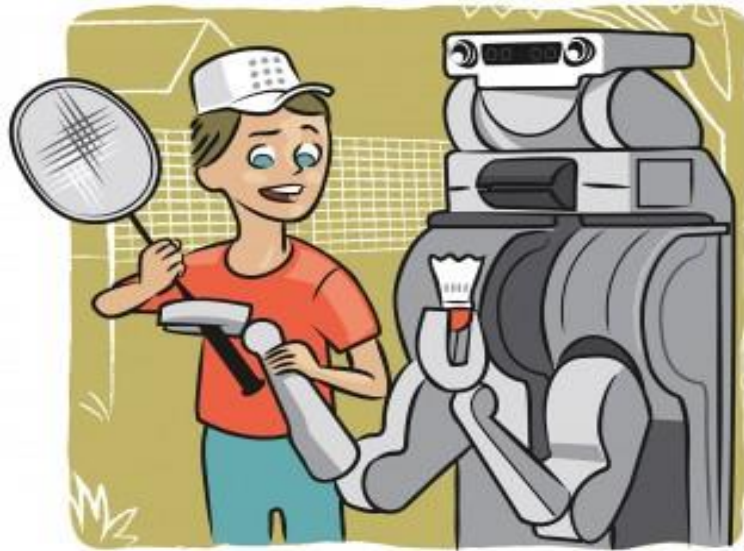


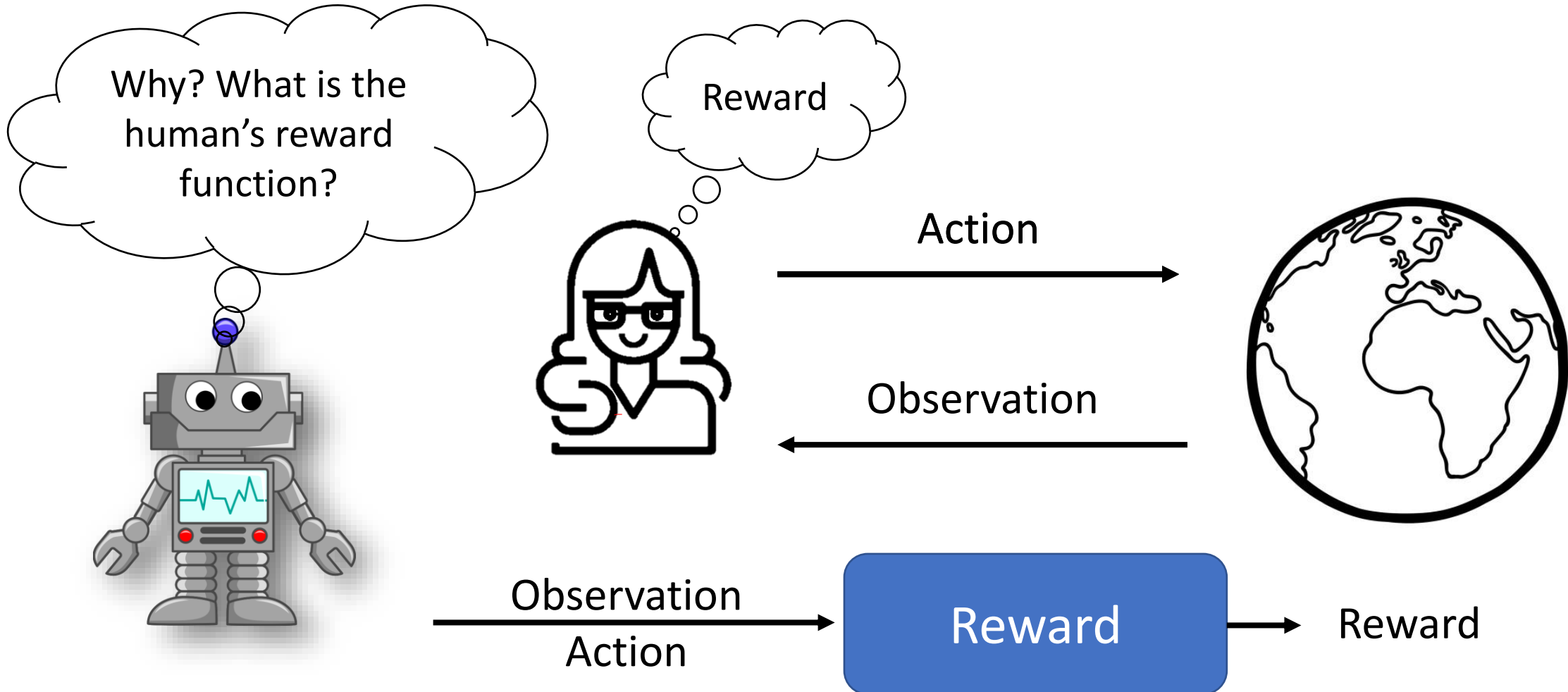
Inverse RL and Reward Learning



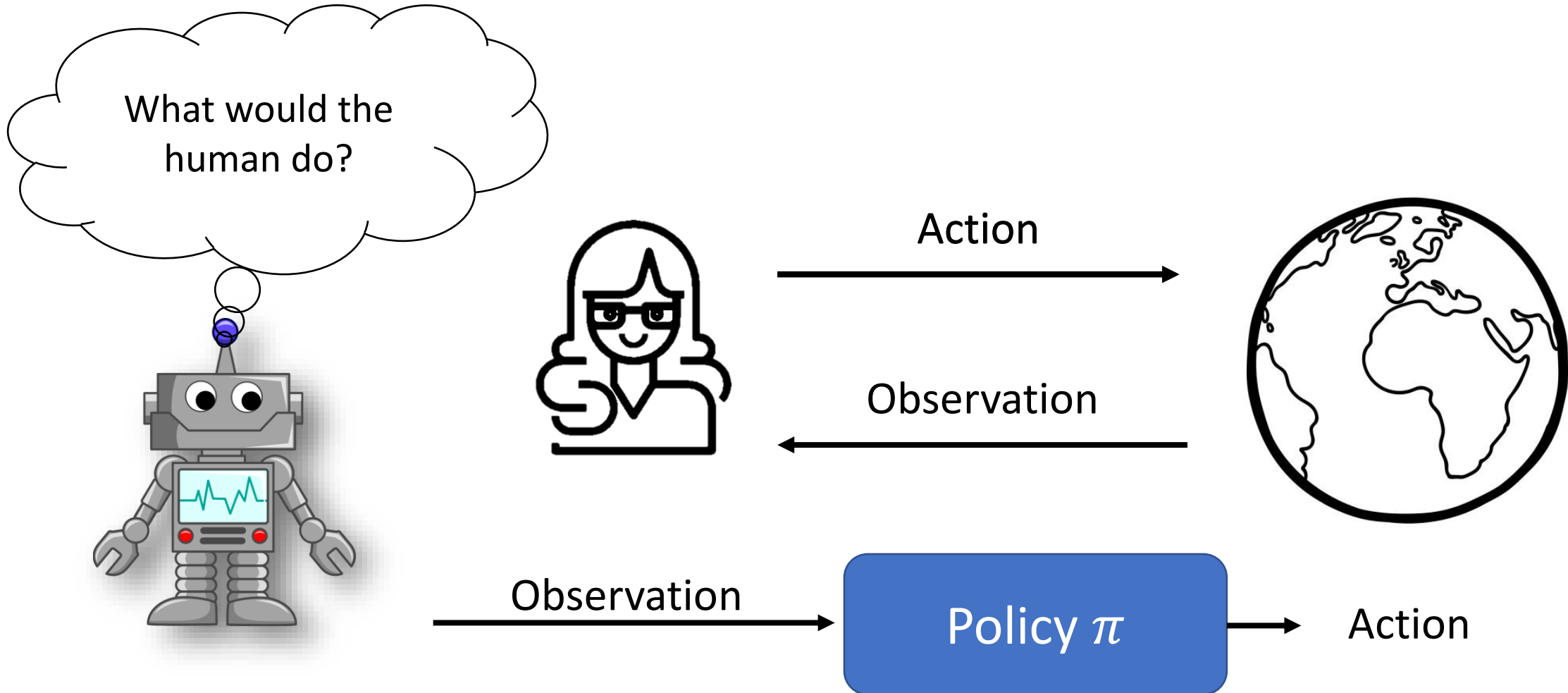
Instructor: Daniel Brown

[Some slides adapted from Sergey Levine (CS 285) and Alina Vereshchaka (CSE4/510)]

Reward Learning (Inverse Reinforcement Learning)



Why not just imitate behavior? (Behavioral Cloning)

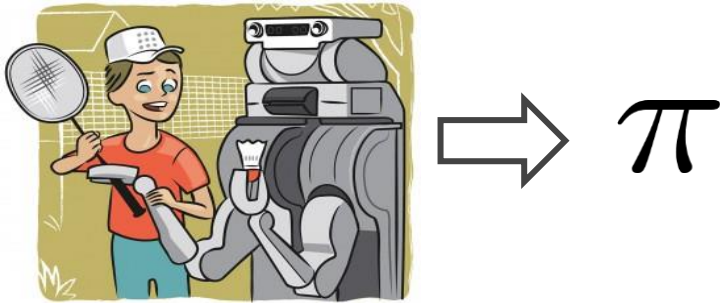


Inverse Reinforcement Learning

- Given
 - MDP without a reward function
 - Demonstrations from an optimal policy π^*
- Recover the reward function R that makes π^* optimal

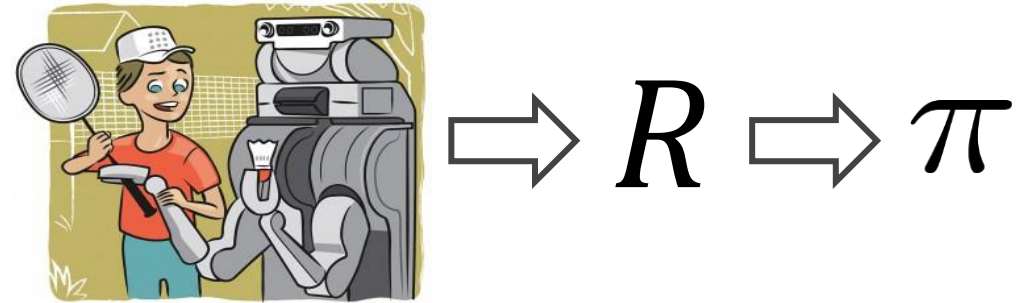
Imitation Learning

Behavioral Cloning



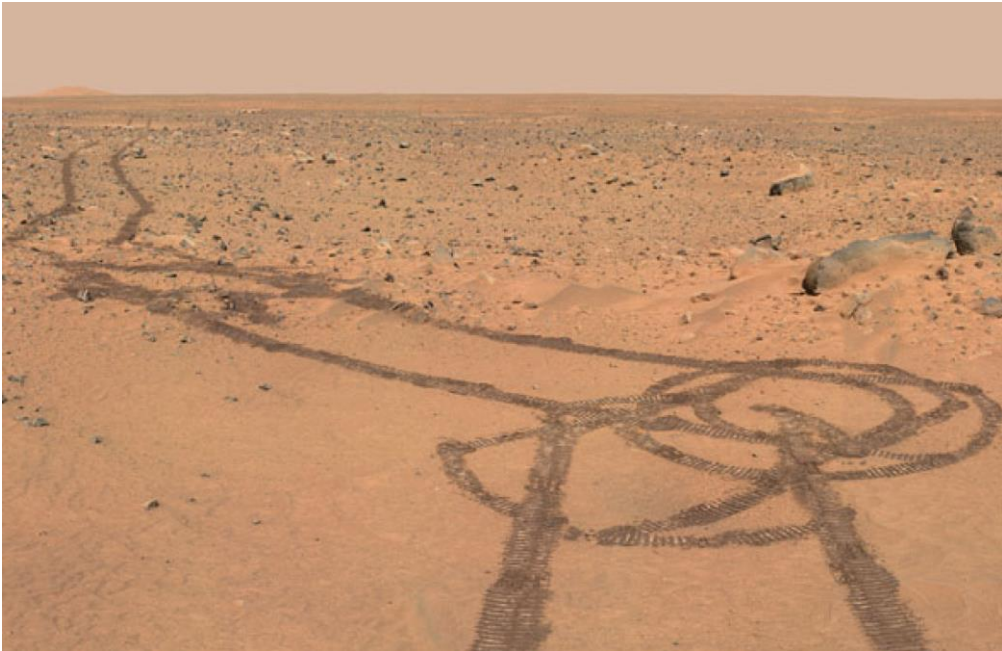
- Answers the “How?” question
- Mimic the demonstrator
- Learn mapping from states to actions
- Computationally efficient
- Compounding errors

Inverse Reinforcement Learning

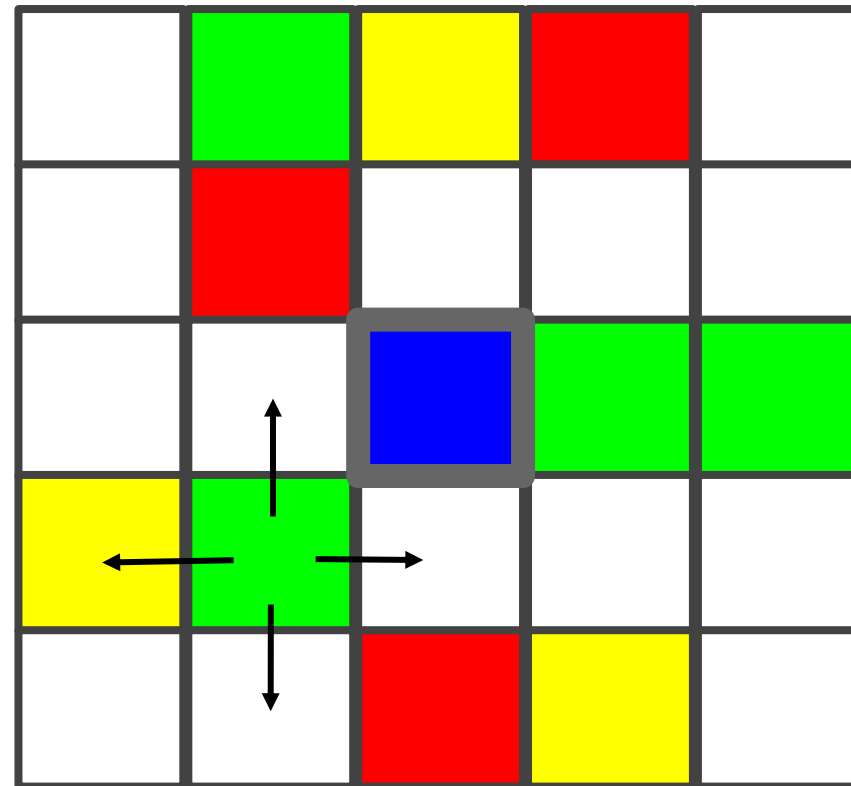


- Answers the “Why?” question
- Explain the demonstrator’s behavior
- Learn a reward function capturing the demonstrator’s intent
- Can require lots of data and compute
- Better generalization. Can recover from arbitrary states

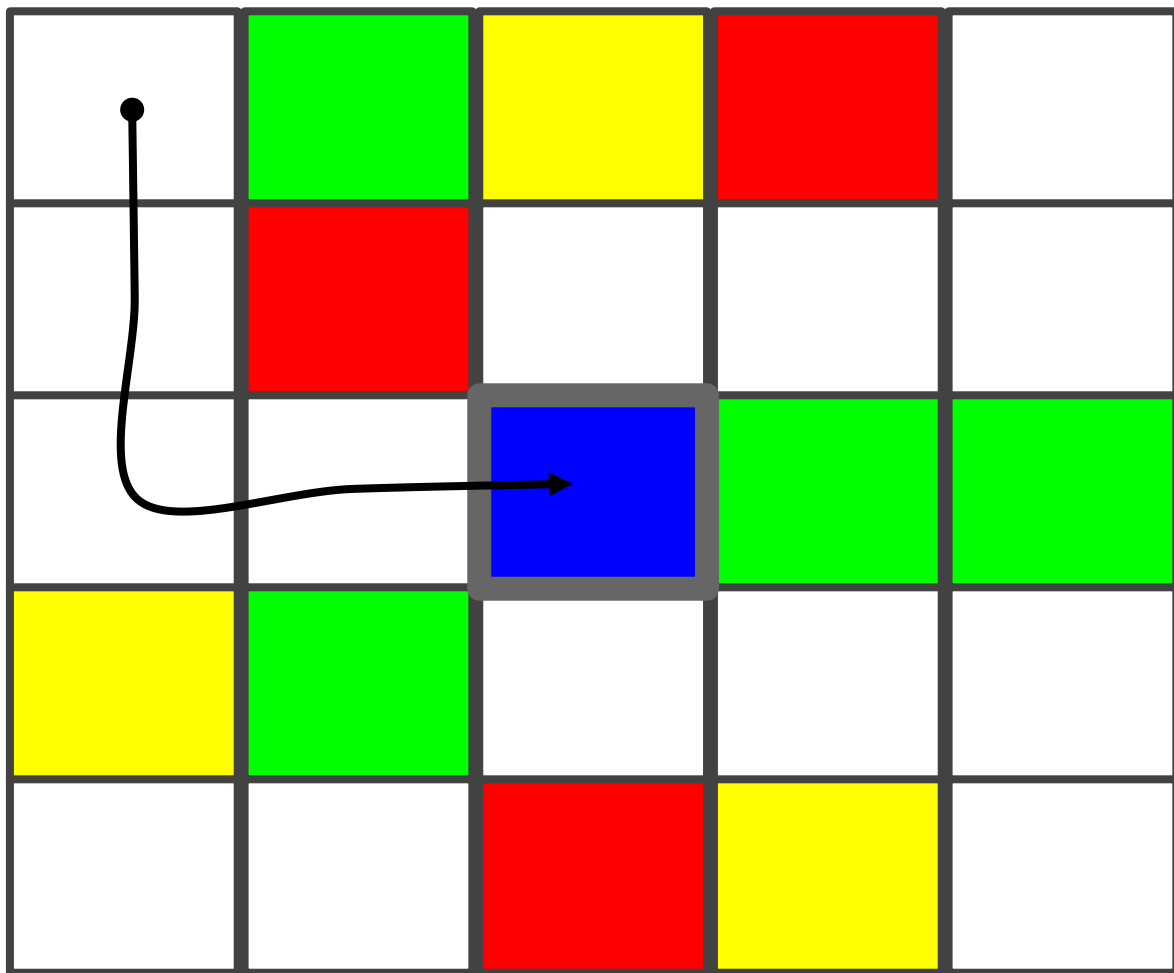
IRL Example: Teaching a robot to navigate through demonstrations



Toy version



What is the reward?



$R(\text{Blue}) = ?$

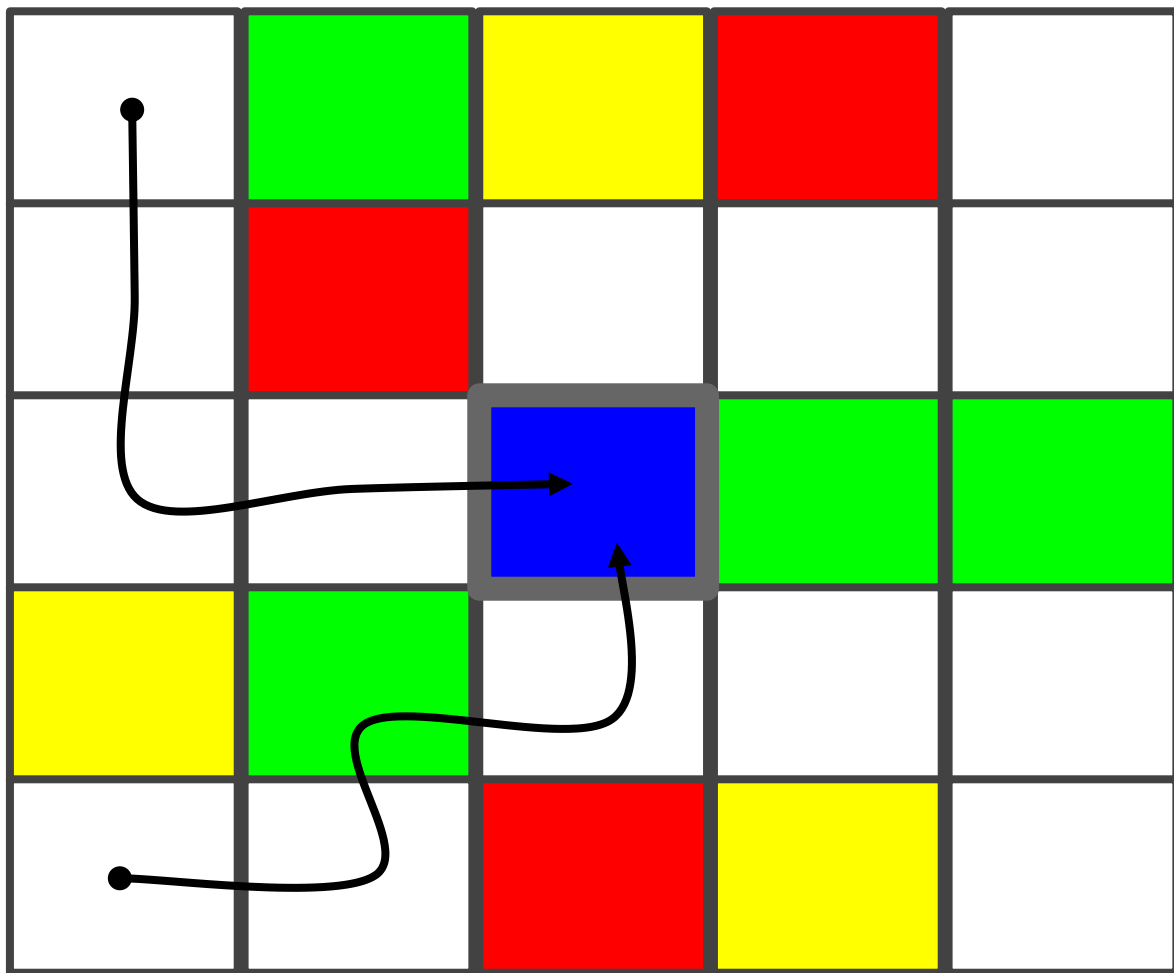
$R(\text{Yellow}) = ?$

$R(\text{Green}) = ?$

$R(\text{Red}) = ?$

$R(\text{White}) = ?$

What is the reward?



$R(\text{Blue Square}) = ?$

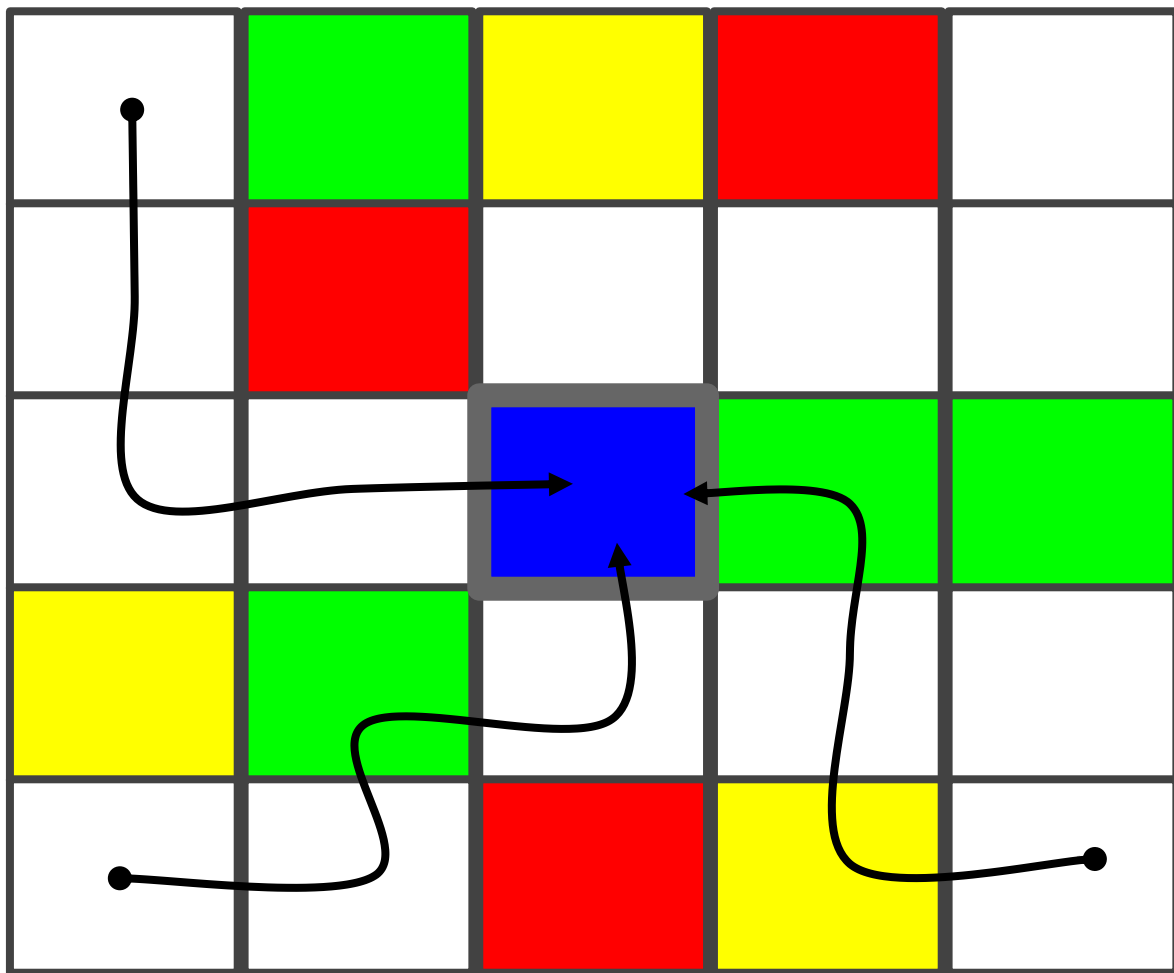
$R(\text{Yellow Square}) = ?$

$R(\text{Green Square}) = ?$

$R(\text{Red Square}) = ?$

$R(\text{White Square}) = ?$

What is the reward?



$R(\text{blue square}) = ?$

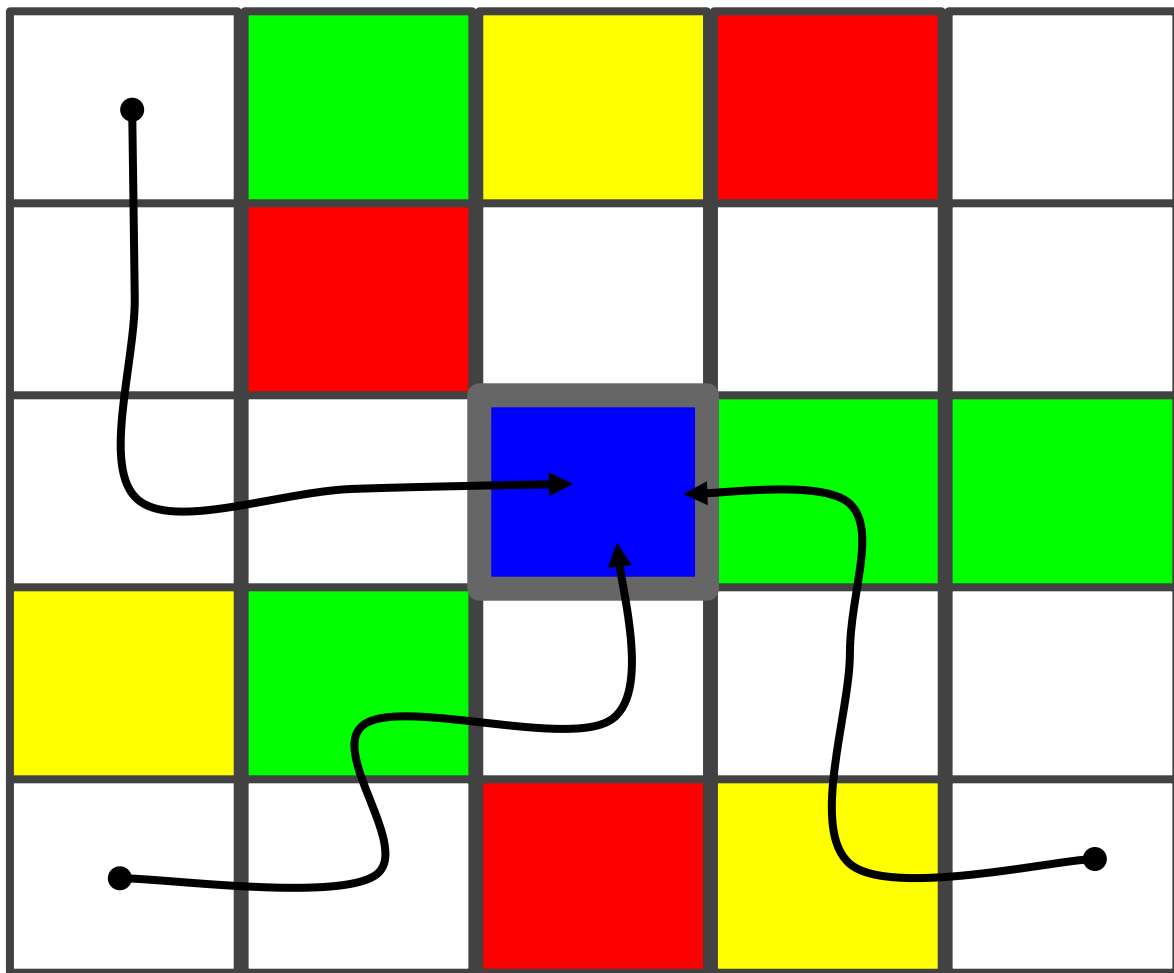
$R(\text{yellow square}) = ?$

$R(\text{green square}) = ?$

$R(\text{red square}) = ?$

$R(\text{empty square}) = ?$

What is the reward?



$$R(\text{Blue Square}) = +1$$

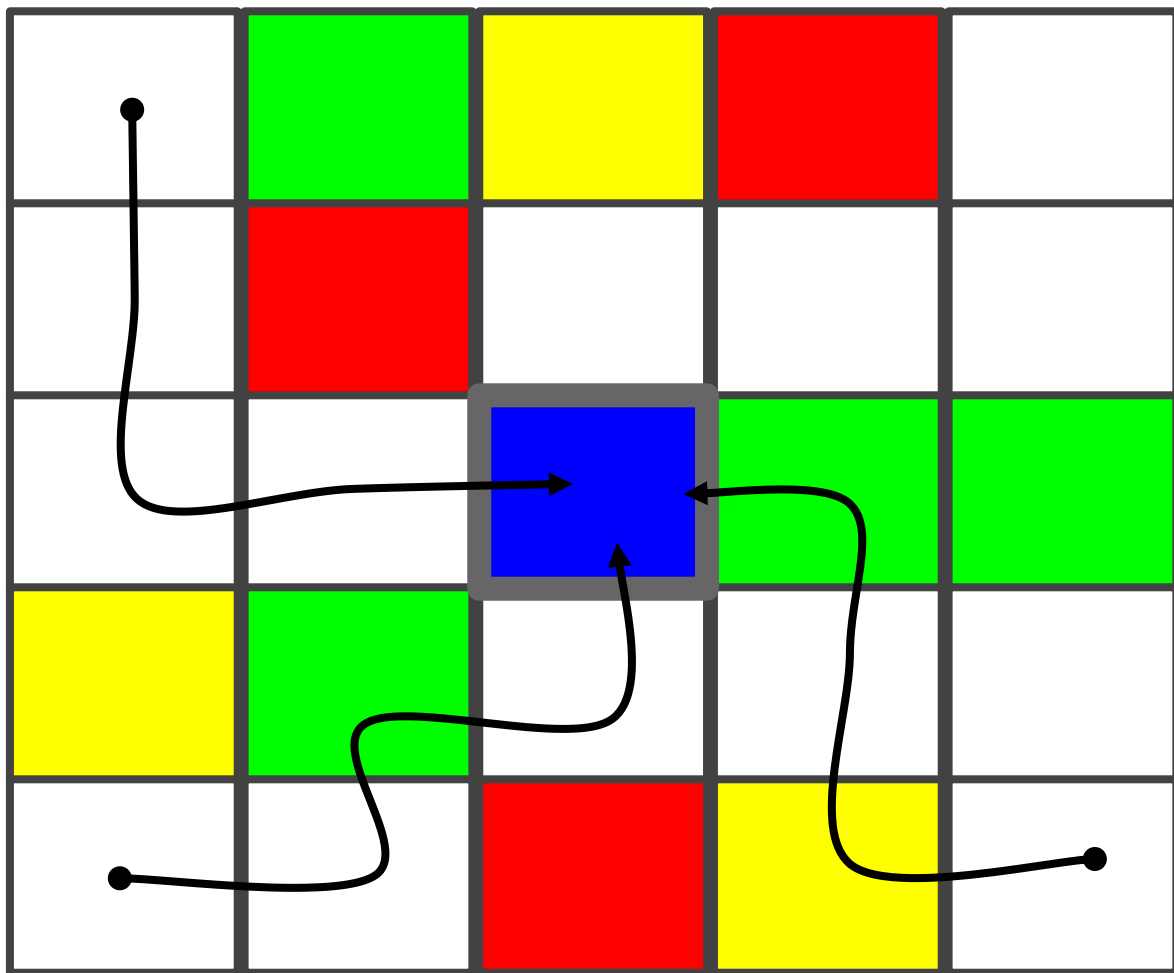
$$R(\text{Yellow Square}) = 0$$

$$R(\text{Green Square}) = 0$$

$$R(\text{Red Square}) = -1$$

$$R(\text{White Square}) = 0$$

What is the reward?



$$R(\text{Blue}) = +10$$

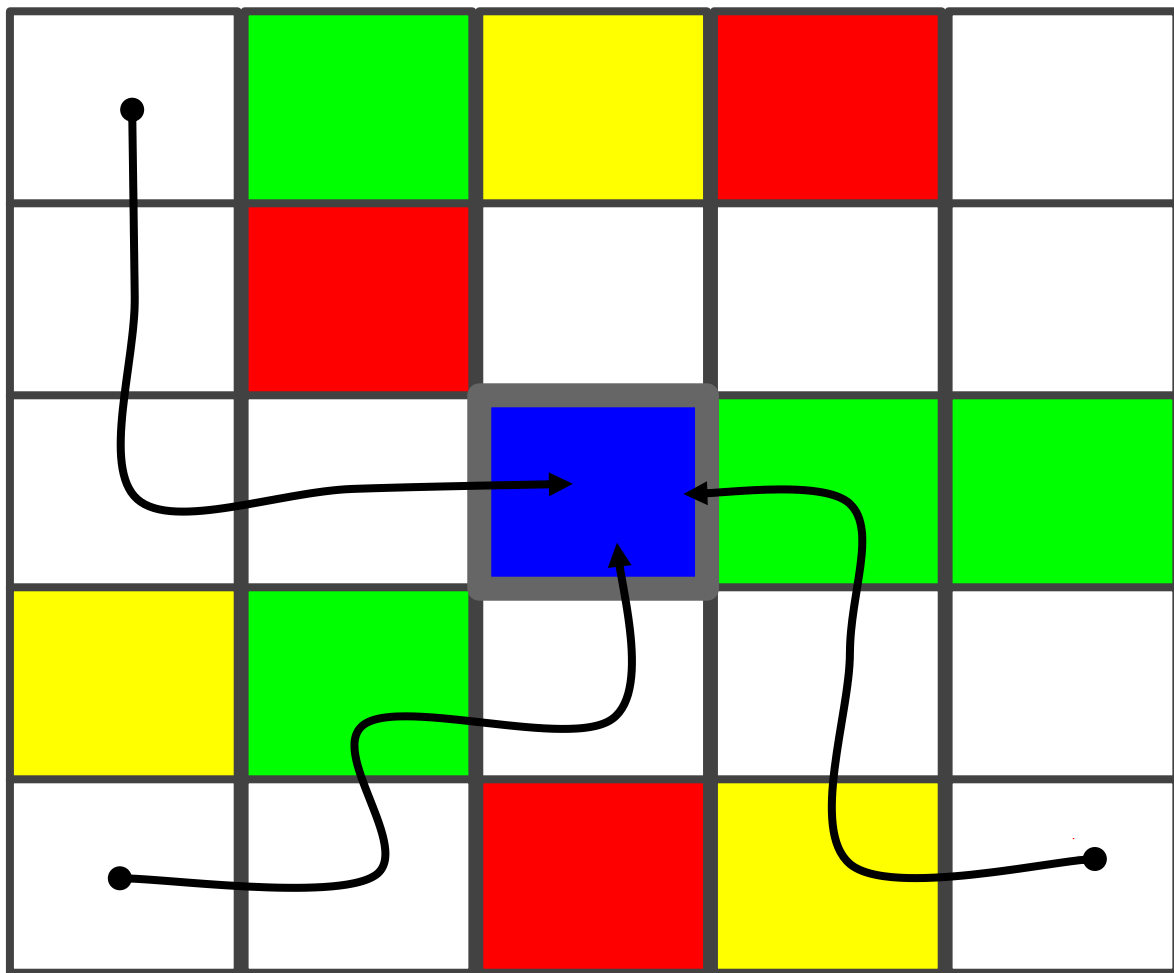
$$R(\text{Yellow}) = 0$$

$$R(\text{Green}) = 0$$

$$R(\text{Red}) = -10$$

$$R(\text{White}) = 0$$

What is the reward?



$$R(\text{Blue Cell}) = +10$$

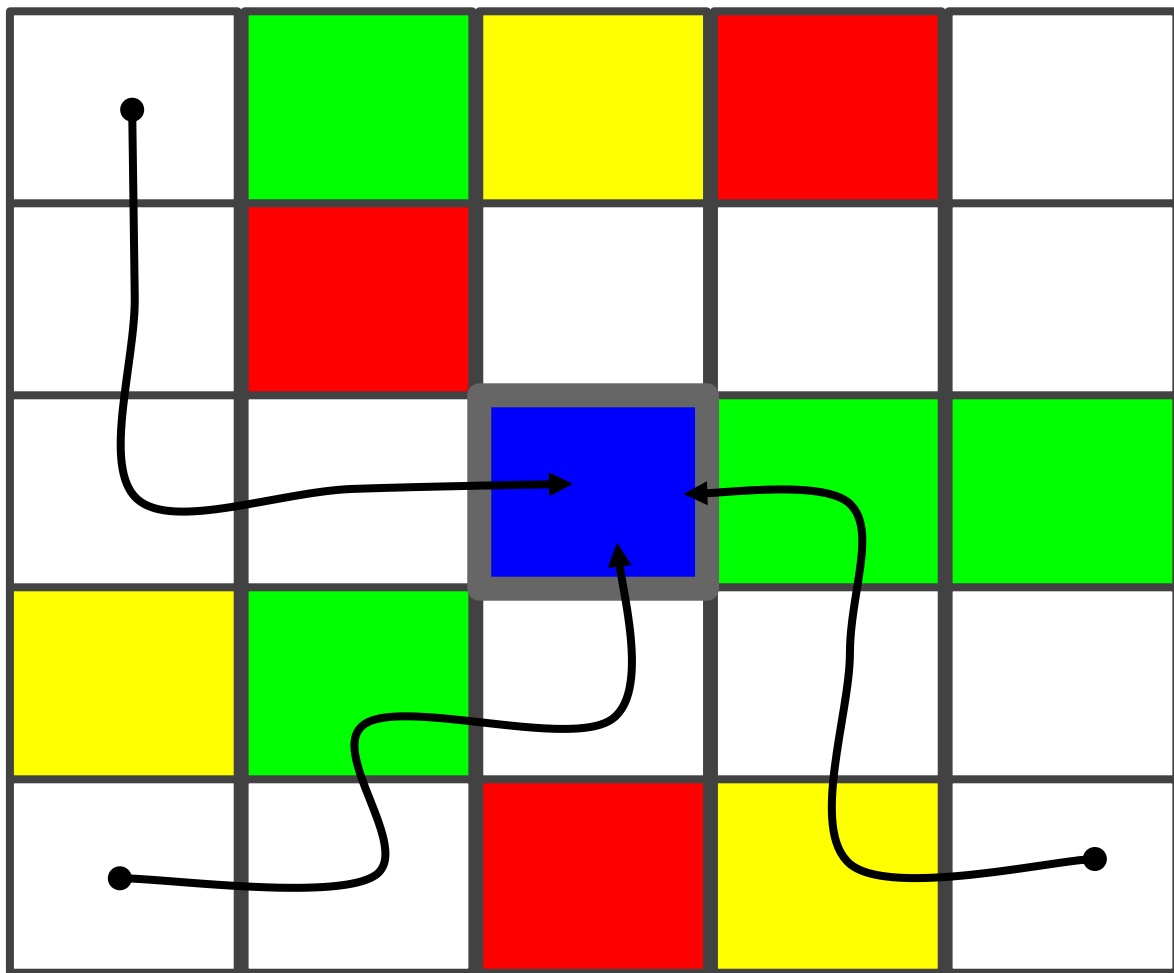
$$R(\text{Yellow Cell}) = -1$$

$$R(\text{Green Cell}) = -1$$

$$R(\text{Red Cell}) = -10$$

$$R(\text{White Cell}) = -1$$

What is the reward?



$$R(\text{Blue Cell}) = 0$$

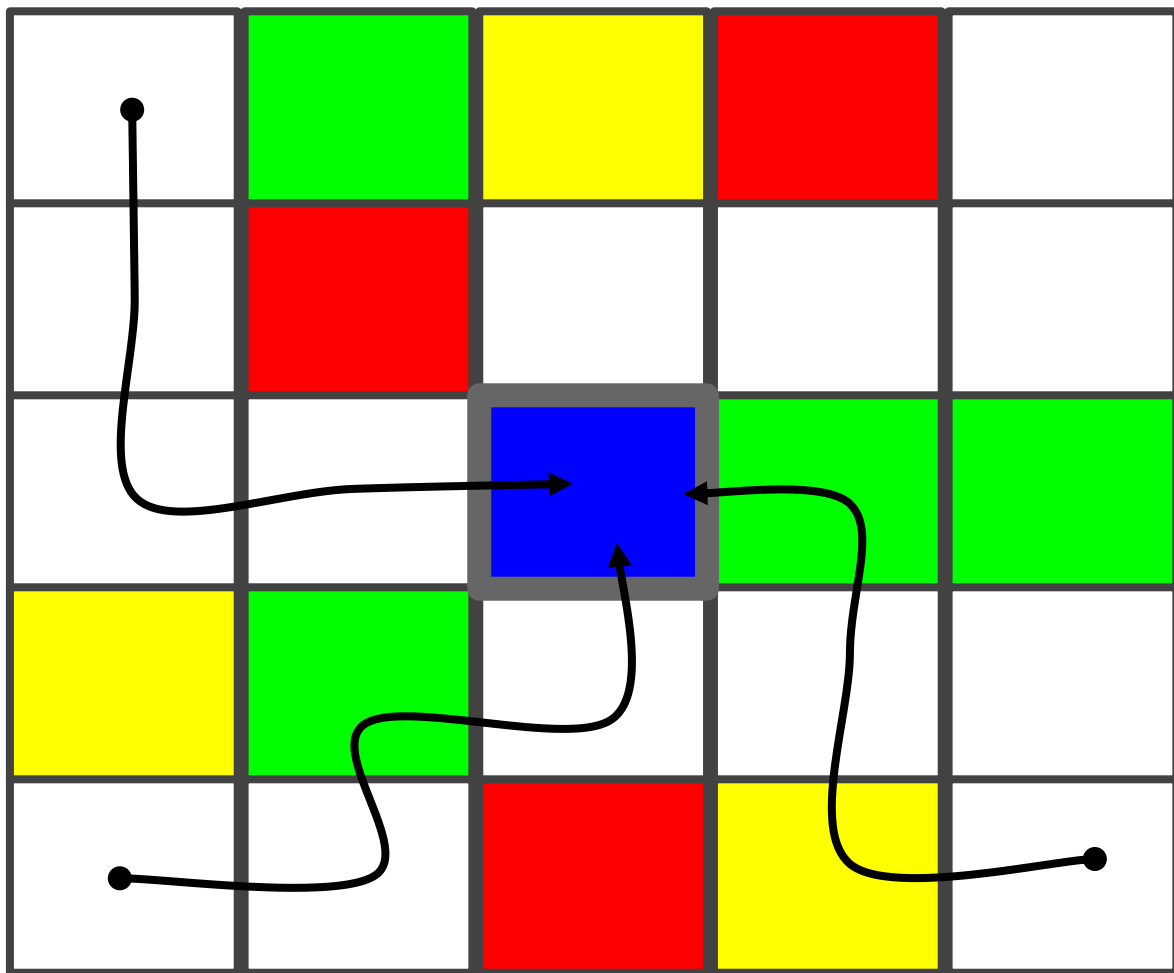
$$R(\text{Yellow Cell}) = 0$$

$$R(\text{Green Cell}) = 0$$

$$R(\text{Red Cell}) = 0$$

$$R(\text{White Cell}) = 0$$

What is the reward?



$$R(\text{Blue Square}) = c$$

$$R(\text{Yellow Square}) = c$$

$$R(\text{Green Square}) = c$$

$$R(\text{Red Square}) = c$$

$$R(\text{White Square}) = c$$

Inverse Reinforcement Learning Formalism

- Given
 - MDP without a reward function
 - Demonstrations from an optimal policy π^*
- Recover a reward function R that makes π^* optimal
- Ill-Posed Problem
 - Infinite number of reward functions that can make π^* optimal
 - Trivial all zero reward
 - Constant reward
 - $aR + c$ (positive scaling $a>0$, and affine shifts)

Simpler problem: What if you know the policy?

How would you do this more generally?

Basic IRL Algorithm

- Start with demonstrations, D
- Guess initial reward function R_0
- $\hat{R} = R_0$
- Loop:
 - Solve for optimal policy $\pi_{\hat{R}}^*$
 - Compare D and $\pi_{\hat{R}}^*$
 - Update $\hat{R}$ to try and make D and $\pi_{\hat{R}}^*$ more similar

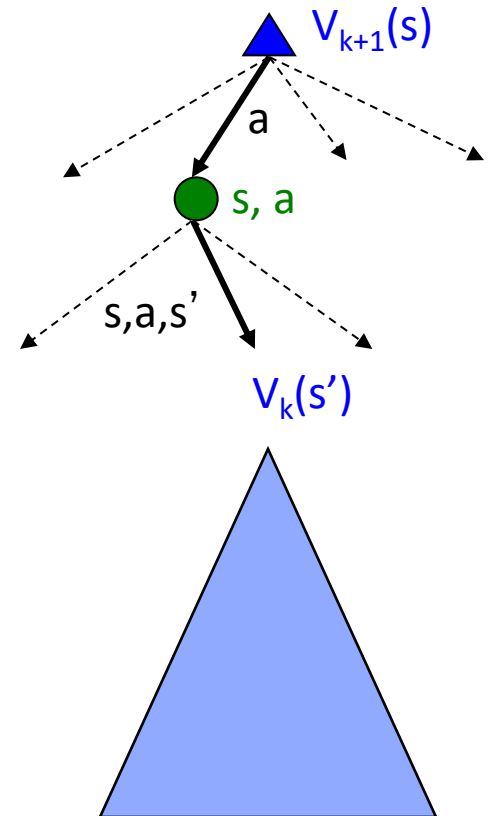
Value Iteration

- Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero
- Given vector of $V_k(s)$ values, do one ply of expectimax from each state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Bellman Update Equation

- Repeat until convergence
- Complexity of each iteration: $O(S^2A)$
- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do



Maximum Entropy IRL (Ziebart et al. 2008)

$$P(\tau) = \frac{e^{R_w(\tau)}}{Z}$$

- Collect M demonstrations $D = \{\tau_1, \dots, \tau_M\}$
- Initialize reward weights $\mathbf{w}$

$$R(s) = \mathbf{w}^T \phi(s)$$

- **Loop**

- Solve for (soft) optimal policy $\pi(a|s)$ via Value Iteration
- Solve for expected feature counts of $\pi(a|s)$
- Compute weight update $\mathbf{w} \leftarrow \mathbf{w} + \alpha(\mu_D - \mu_\pi)$

Soft Value Iteration

$$\pi_{\Theta}(A_t|S_t) = e^{Q_{\pi_{\Theta}}^{\text{soft}}(A_t, S_t) - V_{\pi_{\Theta}}^{\text{soft}}(S_t)}$$

Policy is a softmax policy.

$$V_{\pi_{\Theta}}^{\text{soft}}(S_t) = \log \sum_{A_t \in \mathcal{A}} e^{Q_{\pi_{\Theta}}^{\text{soft}}(A_t, S_t)}$$



Soft Maximum

Soft Value Iteration

$$\pi_{\Theta}(A_t|S_t) = e^{Q_{\pi_{\Theta}}^{\text{soft}}(A_t, S_t) - V_{\pi_{\Theta}}^{\text{soft}}(S_t)}$$

Soft Maximum

$$V_{\pi_{\Theta}}^{\text{soft}}(S_t) = \log \sum_{A_t \in \mathcal{A}} e^{Q_{\pi_{\Theta}}^{\text{soft}}(A_t, S_t)}$$


$$Q_{\pi_{\Theta}}^{\text{soft}}(A_t, S_t) = R_{\Theta}(S_t, A_t) + \sum_{S' \in \mathcal{S}} P_T(S'|A_t, S_t) V_{\pi_{\Theta}}^{\text{soft}}(S')$$

- Initialize values
- Repeat until convergence:
 - Solve for Q
 - Solve for V

Compute expected occupancies

- Given $D_0(s)$
- For $t = 0:N-1$
 - $D_{t+1}(s') = \sum_s D_t(s) \sum_a \pi_t(a|s) T(s'|s, a)$
- $D(s) = \sum_{t=0}^N D_t(s)$

Gradient:

- $\nabla L(\theta) = \tilde{f} - \sum_s D(s) f(s)$

Watch This: Scalable Cost-Function Learning for Path Planning in Urban Environments

Markus Wulfmeier¹, Dominic Zeng Wang¹ and Ingmar Posner¹

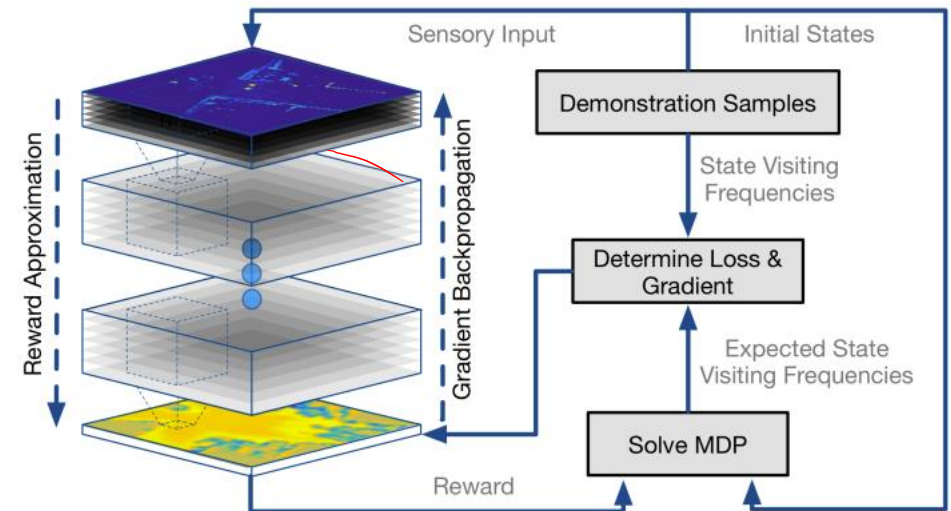
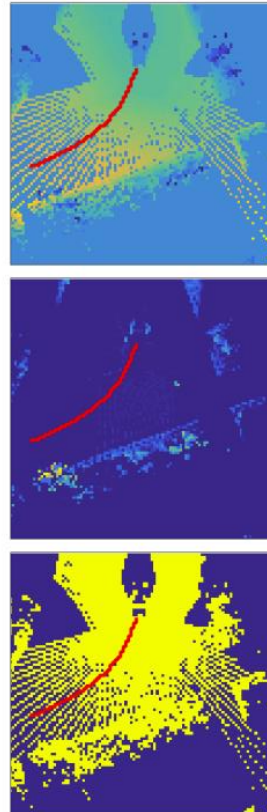


Fig. 1: Schema for training neural networks in the Maximum Entropy paradigm for IRL.

Another way to look at MaxEnt IRL

$$P(\tau) = \frac{e^{R_w(\tau)}}{Z} \quad Z = \int e^{R_w(\tau)} d\tau$$

- Maximum Likelihood Estimation
- Find reward function that maximizes the log likelihood of the demonstration trajectories:

$$\max_{\theta} \frac{1}{N} \sum_{\tau \in D} R_w(\tau) - \log Z$$

How to avoid fully solving MDP

$$\max_{\theta} \frac{1}{N} \sum_{\tau \in D} R_w(\tau) - \log Z \quad Z = \int e^{R_w(\tau)} d\tau$$

- Estimate Z with a finite set of trajectories Z_τ .
- Loop:
 - Update parameters w so demonstrations have higher reward than trajectories in Z_τ .
 - Update Z_τ

How to make this more tractable

Relative Entropy Inverse Reinforcement Learning

Abdeslam Boularias

Jens Kober

Jan Peters

Max-Planck Institute for Intelligent Systems

72076 Tübingen, Germany

{abdeslam.boularias,jens.kober,jan.peters}@tuebingen.mpg.de

$$P(\tau) = \frac{e^{R_w(\tau)}}{Z}$$

Uniform sampling to approximate Z .

Learning Objective Functions for Manipulation

Mrinal Kalakrishnan*, Peter Pastor*, Ludovic Righetti*[†], and Stefan Schaal*[†]

kalakris@usc.edu, pastorsa@usc.edu, ludovic.righetti@a3.epfl.ch, sschaal@usc.edu

*CLMC Lab, University of Southern California, Los Angeles CA 90089

[†]Max Planck Institute for Intelligent Systems, Tübingen, Germany 72076

Noisy perturbations of demonstrations to approximate Z

Guided Cost Learning: Deep Inverse Optimal Control via Policy Optimization

Chelsea Finn
Sergey Levine
Pieter Abbeel

University of California, Berkeley, Berkeley, CA 94709 USA

CBFINN@EECS.BERKELEY.EDU
SVLEVINE@EECS.BERKELEY.EDU
PABBEEL@EECS.BERKELEY.EDU

Use current policy to approximate Z .
Alternate between a few steps of reward updates and a few steps of policy updates.

Finn et al. "Guided Cost Learning." 2016

